1. Variables and their size

1. `int` 4 bytes
2. `long` 8 bytes
3. `char` 2 bytes
4. `boolean` 1 bit
5. `double` 8 bytes

1.1 Integer overflow

When left and right are both above 1 billion, their sum exceeds int range and wraps to a negative number. The safe version avoids this by computing the difference first.

```
// BUG: if left + right > Integer.MAX_VALUE, this overflows
int mid = (left + right) / 2;

// CORRECT: safe from overflow
int mid = left + (right - left) / 2;
```

Note: 1_000_000 and 1000000 are same, its just underscore has better readability.

2. Bitwise operators

```
// XOR trick: find the number that appears once (all others appear twice)
// nums = [4, 1, 2, 1, 2]
int single = 0;
for (int num : nums) {
    single ^= num;  // XOR cancels out duplicates
}
// single = 4
```

Symbol	Use
OR	Setting bits: n OR (1 << i)
& AND	Masking, checking if bit is set: (n & (1 << i)) != 0

Symbol	Use
^ XOR	Finding unique elements, toggling bits
~ NOT	inversion
<<	Multiply by powers of 2: 1 << n mean 2^n
>>	Divide by powers of 2

```
// Check if bit i is set
boolean isSet = (mask & (1 << i)) != 0;

// Set bit i
mask = mask | (1 << i);

// Clear bit i
mask = mask & ~(1 << i);

// Toggle bit i
mask = mask ^ (1 << i);

// Count number of set bits
int count = Integer.bitCount(mask);
```

3. Default values

1. int[] - 0
2. long[] - 0L
3. boolean[] - false
4. char[] - '\0' (null char)
5. Object[] - null

3.1 Length/Size

```java
int[] arr = {1, 2, 3};
arr.length      // 3 (property, no parentheses)

String s = "hello";
s.length()      // 5 (method, with parentheses)

List<Integer> list = new ArrayList<>();
list.size()     // 0 (method, with parentheses)
```

3.2 Arrays for graphs grid/matrix

```java
int[][] matrix = new int[rows][cols];
```

3.3 DP grid

```java
int[] dp = new int[n + 1];
int[][] dp = new int[m + 1][n + 1];
```

3.3 4-directional movement in grid

```java
int[][] dirs = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}}; // up-down-left-right

for (int[] dir : dirs) {
    int newRow = row + dir[0];
    int newCol = col + dir[1];
    if (newRow >= 0 && newRow < rows && newCol >= 0 && newCol < cols) {
        // Process neighbor
    }
}
```

3.4 Arrays util class methods

```
Arrays.sort(nums);
Arrays.fill(arr, 0);
int[] copy = Arrays.copyOfRange(nums, 1, 4);// partial or full

List<String> words = Arrays.asList("hello", "world"); // you can not add or remove to w
//for that you need to do this
List<String> mutable = new ArrayList<>(Arrays.asList("a", "b"));
```

4. Collection Framework and Strings

Strings in java are immutable and when you concatenate something, it creates a new object everytime, therefore use StringBuilder for this, if want to modify something

```
StringBuilder sb = new StringBuilder();
for(char c: chars){
    sb.append(c);
}
return sb.toString();
```

Method	Use
charAt(i)	Access individual characters
toCharArray()	When you need in-place modification
indexOf(str)	Find substrings (-1 if not found)
equals(other)	Content comparison
isEmpty()	Check if length is 0
contains(seq)	substring check
split(regex)	tokenize strings

4.1 Equality check in strings

```java
String a = new String("hello");
String b = new String("hello");

a == b       // false (compares references, different objects)
a.equals(b)  // true (compares content)
```

4.2 StringBuilder methods

```java
StringBuilder sb = new StringBuilder();
sb.append("hello");      // Add to end
sb.append(' ');           // Append char
sb.insert(0, "say ");    // Insert at position
sb.reverse();             // Reverse in place
sb.deleteCharAt(0);       // Remove character
sb.setCharAt(2, 'x');    // Replace character at index
sb.length();              // Current length
String result = sb.toString(); // Convert back to String
```

4.3 Character util methods

```java
Character.isLetterOrDigit(c)  // Alphanumeric check (valid palindrome)
Character.isLetter(c)          // Letter only
Character.isDigit(c)           // Digit only
Character.isUpperCase(c)       // Uppercase check
Character.toLowerCase(c)       // Case conversion
Character.toUpperCase(c)       // Case conversion
c - 'a'                        // Map 'a'-'z' to 0-25 (frequency arrays)
c - '0'                        // Map '0'-'9' to 0-9 (digit extraction)
(char) ('a' + index)           // Map 0-25 back to 'a'-'z'
```

`c-'a'` is how you build freq maps without using hashmaps, more faster and efficient than hashmap

```java
// Frequency array for lowercase letters (26 slots)
int[] freq = new int[26];
for (char c : s.toCharArray()) {
    freq[c - 'a']++;
}
// freq[0] = count of 'a', freq[1] = count of 'b', etc.
```

4.4 ArrayList util methods

```java
List<Integer> result = new ArrayList<>();
result.add(42);          // Append: O(1) amortized
result.get(0);           // Access by index: O(1)
result.set(0, 99);       // Update by index: O(1)
result.size();           // Current size
result.isEmpty();        // Check if empty
result.contains(42);     // Search: O(n)
result.remove(0);        // Remove by index: O(n) (shifts elements)
result.remove(Integer.valueOf(42)); // Remove by value: O(n) first occurance of 42
```

4.5 Collections class

```java
Collections.reverse(list);                // Reverse in place
Collections.sort(list);                   // Sort ascending
Collections.sort(list, Collections.reverseOrder()); // Sort descending
Collections.swap(list, i, j);            // Swap two elements
Collections.min(list);                    // Find minimum
Collections.max(list);                    // Find maximum
Collections.frequency(list, element);     // Count occurrences
```

4.6 HashMap and HashSet

```java
// Frequency counting -- the most common HashMap pattern
Map<Character, Integer> freq = new HashMap<>();
for (char c : s.toCharArray()) {
    freq.put(c, freq.getOrDefault(c, 0) + 1);
}

// Check existence and retrieve
if (freq.containsKey('a')) {
    int count = freq.get('a');
}

// Safe retrieval (avoids NullPointerException)
int count = freq.getOrDefault('z', 0);

// Iterate over all entries
for (Map.Entry<Character, Integer> entry : freq.entrySet()) {
    char key = entry.getKey();
    int value = entry.getValue();
}

// Iterate over keys only
for (Character key : freq.keySet()) { ... }

// Iterate over values only
for (Integer value : freq.values()) { ... }

// Remove a key
freq.remove('a');

// Get size
freq.size();


// for HashSet java
Set<Integer> visited = new HashSet<>();
visited.add(node);           // Returns true if newly added, false if already present
visited.contains(node);      // O(1) lookup
visited.remove(node);        // O(1) removal
visited.size();              // Number of elements
```

4.7 LinkedhashMap and LinkedHashSet

LinkedHashMap maintains insertion order, which is important for specific problems like LRU Cache

```
// Simple LRU cache using LinkedHashMap's access-order mode
Map<Integer, Integer> cache = new LinkedHashMap<>(16, 0.75f, true);
// The 'true' parameter enables access-order (most recently accessed moves to end)
```

4.8 TreeMap and TreeSet

```
TreeMap<Integer, Integer> map = new TreeMap<>();
map.put(5, 10);
map.put(1, 20);
map.put(10, 30);

map.firstKey();              // 1 (smallest key)
map.lastKey();               // 10 (largest key)
map.floorKey(7);             // 5 (largest key <= 7)
map.ceilingKey(7);           // 10 (smallest key >= 7)
map.lowerKey(5);             // 1 (largest key strictly < 5)
map.higherKey(5);            // 10 (smallest key strictly > 5)
map.headMap(5);              // All entries with key < 5
map.tailMap(5);              // All entries with key >= 5
map.subMap(1, 10);           // All entries with 1 <= key < 10
```

4.9 Queue with LL (must for BFS)

```
Queue<int[]> queue = new LinkedList<>();
queue.offer(new int[]{startRow, startCol}); // Enqueue
while (!queue.isEmpty()) {
    int size = queue.size();  // Capture size for level-order traversal
    for (int i = 0; i < size; i++) {
        int[] cell = queue.poll();           // Dequeue
        // Process cell, add neighbors
        queue.offer(new int[]{newRow, newCol});
    }
}
```

4.10 ArrayDeque

Never use Stack in Java. It extends Vector, which is synchronized (slow) and a legacy class. Use ArrayDeque instead:

```
// As a stack (LIFO)
Deque<Integer> stack = new ArrayDeque<>();
stack.push(42);      // Push to top
stack.pop();         // Remove and return top (throws if empty)
stack.peek();        // View top without removing (returns null if empty)
stack.isEmpty();     // Check before pop to avoid exception

// As a double-ended queue (sliding window maximum, monotonic deque)
Deque<Integer> deque = new ArrayDeque<>();
deque.offerFirst(val);   // Add to front
deque.offerLast(val);    // Add to back
deque.pollFirst();       // Remove from front (returns null if empty)
deque.pollLast();        // Remove from back (returns null if empty)
deque.peekFirst();       // View front
deque.peekLast();        // View back
```

4.11 PriorityQueue

PriorityQueue gives you a min-heap by default. It is essential for problems involving top-K elements, merge-K-sorted, and Dijkstra's algorithm

```
// Min-heap (default): smallest element comes out first
PriorityQueue<Integer> minHeap = new PriorityQueue<>();

// Max-heap: largest element comes out first
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());

// Custom comparator: sort by first element of array
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[0], b[0]));

pq.offer(element);   // Insert: O(log n)
pq.poll();           // Remove min/max: O(log n)
pq.peek();           // View min/max: O(1)
pq.size();           // Current size
pq.isEmpty();        // Check if empty
```

5. Sorting and Comparators

```java
// Sort primitive array
int[] nums = {3, 1, 4, 1, 5};
Arrays.sort(nums); // [1, 1, 3, 4, 5]

// Sort a portion of an array (index 1 to 3, exclusive)
Arrays.sort(nums, 1, 4);

// Sort a list
List<Integer> list = new ArrayList<>(Arrays.asList(3, 1, 4));
Collections.sort(list);
// Or equivalently:
list.sort(Comparator.naturalOrder());
```

5.1 Custom Sorting with Comparator

for custom sorting, java uses the Comparator class with lambda expressions

```java
// Sort intervals by start time (merge intervals pattern)
int[][] intervals = {{1,3}, {2,6}, {8,10}};
Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

// Sort strings by length, then alphabetically for ties
Arrays.sort(words, (a, b) -> {
    if (a.length() != b.length()) return Integer.compare(a.length(), b.length());
    return a.compareTo(b);
});

// Equivalent using Comparator chaining (cleaner for multi-key sorts)
Arrays.sort(words, Comparator.comparingInt(String::length).thenComparing(Comparator.nat
```

5.2 for Descending Sorting

```java
// Sort Integer array in descending order
Integer[] nums = {3, 1, 4};
Arrays.sort(nums, Collections.reverseOrder());

// Sort list in descending order
list.sort(Collections.reverseOrder());
```

6. Functional Patterns for DSA

```java
// 1. getOrDefault: safe frequency counting
map.getOrDefault(key, 0);  // Returns 0 if key not found

// 2. computeIfAbsent: build adjacency lists cleanly
Map<Integer, List<Integer>> graph = new HashMap<>();
graph.computeIfAbsent(node, k -> new ArrayList<>()).add(neighbor);

// 3. merge: concise frequency counting
map.merge(key, 1, Integer::sum);  // Increment count by 1

// 4. putIfAbsent: set a value only if key is not already present
map.putIfAbsent(key, defaultValue);

// 5. entrySet: iterate over key-value pairs
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    String key = entry.getKey();
    int value = entry.getValue();
}
```

computeIfAbsent is more elegant, see the difference below

```java
// Without computeIfAbsent (verbose)
if (!graph.containsKey(node)) {
    graph.put(node, new ArrayList<>());
}
graph.get(node).add(neighbor);

// With computeIfAbsent (clean)
graph.computeIfAbsent(node, k -> new ArrayList<>()).add(neighbor);
```

7. 3-Rules of Null Handling

null is a source of many runtime errors in Java. In DSA, you encounter it primarily in three situations: tree/linked list problems, map lookups, and uninitialized object arrays.

7.1 Rule 1: Always check for null before accessing fields or calling methods on an object.

```java
// Tree traversal: always check for null children
if (node.left != null) {
    queue.offer(node.left);
}

// Linked list: check before advancing
while (current != null) {
    // process current
    current = current.next;
}
```

7.2 Rule 2: Use `getOrDefault` instead of `get` for maps.

```java
// Dangerous: returns null if key missing, unboxing null throws NPE
int count = map.get("key");

// Safe: returns default value
int count = map.getOrDefault("key", 0);
```

7.3 Rule 3: Combine null and empty checks.

```
if (nums == null || nums.length == 0) return new int[0];
if (s == null || s.isEmpty()) return "";
```

8. Common Problem and Solution

8.1 Iterating and Modifying Collections Safely

One of Java's most common runtime errors is `ConcurrentModificationException` , thrown when you modify a collection while iterating over it with a for-each loop:

```
// WRONG: throws ConcurrentModificationException
List<Integer> list = new ArrayList<>(Arrays.asList(1, 2, 3, 4, 5));
for (int num : list) {
    if (num % 2 == 0) {
        list.remove(Integer.valueOf(num));
    }
}
```

There are three safe alternatives:

```java
// Option 1: Use an Iterator with iterator.remove()
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    if (it.next() % 2 == 0) {
        it.remove(); // Safe removal during iteration
    }
}

// Option 2: Collect items to remove, then remove after the loop
List<Integer> toRemove = new ArrayList<>();
for (int num : list) {
    if (num % 2 == 0) toRemove.add(num);
}
list.removeAll(toRemove);

// Option 3: Iterate backward with index-based loop (for ArrayList)
for (int i = list.size() - 1; i >= 0; i--) {
    if (list.get(i) % 2 == 0) {
        list.remove(i);
    }
}
```

8.2 Recursion and the Call Stack

Recursion is the backbone of tree traversal, graph DFS, backtracking, and divide-and-conquer. Understanding how Java handles recursion is essential.
If your recursion goes too deep, you get a StackOverflowError.

THE FIX: Convert deep recursion to an iterative approach using an explicit stack:

```java
// Recursive DFS (risks stack overflow for deep graphs)
void dfs(int node, boolean[] visited) {
    visited[node] = true;
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) dfs(neighbor, visited);
    }
}

// Iterative DFS (safe for any depth)
void dfs(int start, boolean[] visited) {
    Deque<Integer> stack = new ArrayDeque<>();
    stack.push(start);
    while (!stack.isEmpty()) {
        int node = stack.pop();
        if (visited[node]) continue;
        visited[node] = true;
        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) stack.push(neighbor);
        }
    }
}
```

9. Graph Representation patterns

Graphs appear in a large portion of DSA problems. Java does not have a built-in graph class, so you need to build representations yourself. There are two common approaches:

9.1 Adjacency list with `List<List<Integer>>` -- Use when nodes are numbered 0 to n-1.

pros - Fast index access, no hashing overhead
cons - Wastes space if node IDs are sparse
use when - Nodes are 0 to n-1

```java
int n = 5; // number of nodes
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) {
    graph.add(new ArrayList<>());
}

// Add edges
for (int[] edge : edges) {
    graph.get(edge[0]).add(edge[1]);
    graph.get(edge[1]).add(edge[0]); // Omit for directed graphs
}

// Access neighbors
for (int neighbor : graph.get(node)) { ... }
```

9.2 Adjacency list with `HashMap<Integer, List<Integer>>` -- Use when node IDs are not contiguous or are very large.

pros - Handles any node IDs, no wasted space

cons - Slightly slower due to hashing

use when - Node IDs are large, sparse, or non-numeric

```java
Map<Integer, List<Integer>> graph = new HashMap<>();
for (int[] edge : edges) {
    graph.computeIfAbsent(edge[0], k -> new ArrayList<>()).add(edge[1]);
    graph.computeIfAbsent(edge[1], k -> new ArrayList<>()).add(edge[0]);
}

// Access neighbors (with safety check)
for (int neighbor : graph.getOrDefault(node, Collections.emptyList())) { ... }
```

9.3 for weighted graphs use int[] pairs or a simple class to store neighbor and weight:

```
// Adjacency list for weighted graph: List of [neighbor, weight] pairs
List<List<int[]>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

graph.get(from).add(new int[]{to, weight});

// Used in Dijkstra:
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> Integer.compare(a[1], b[1]));
pq.offer(new int[]{startNode, 0}); // [node, distance]
```

10. Pair and tuple alternatives

Java has no built-in Pair or Tuple class (unlike Python). In DSA problems, you frequently need to store two or three values together.

10.1 `int[]` array (most common in DSA)

```
// Store [row, col] pair
int[] cell = new int[]{row, col};
queue.offer(new int[]{row, col, distance}); // Can store 3+ values

// Access
int r = cell[0];
int c = cell[1];
```

10.2 Custom class (when clarity matters)

```
class Pair {
    int key;
    int value;
    Pair(int key, int value) {
        this.key = key;
        this.value = value;
    }
}
```

11. Deduplication patterns

Many DSA problems require avoiding duplicate results (e.g., 3Sum, 4Sum, permutations with duplicates).

11.1 Sort and skip duplicates (preferred for sorted array problems)

```
Arrays.sort(nums);
for (int i = 0; i < nums.length; i++) {
    // Skip duplicates: if this value is the same as the previous, skip it
    if (i > 0 && nums[i] == nums[i - 1]) continue;
    // Process nums[i]
}
```

11.2 Use a Set to collect unique results

```
Set<List<Integer>> resultSet = new HashSet<>();
// ... generate candidates ...
resultSet.add(Arrays.asList(a, b, c)); // Duplicates are automatically ignored
return new ArrayList<>(resultSet);
```

12. Common DSA idioms

These are the small patterns and utility calls you will reach for repeatedly.

12.1 Swapping elements

```
int temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;
```

12.2 Sentinel values for tracking min/max

```
int minVal = Integer.MAX_VALUE;  // 2,147,483,647
int maxVal = Integer.MIN_VALUE;  // -2,147,483,648
for (int num : nums) {
    minVal = Math.min(minVal, num);
    maxVal = Math.max(maxVal, num);
}
```

12.3 Math utils

```
Math.max(a, b);              // Larger of two values
Math.min(a, b);              // Smaller of two values
Math.abs(a);                 // Absolute value (careful with MIN_VALUE)
(int) Math.pow(2, 10);      // Power (returns double, cast to int)
Math.sqrt(n);                // Square root (returns double)
Math.log(n);                 // Natural logarithm
Math.ceil(a / (double) b);  // Ceiling division (cast to double first!)
```

12.4 Ceiling division

```
// Integer ceiling division: ceil(a / b) for positive a, b
int ceil = (a + b - 1) / b;
// Or equivalently:
int ceil = (a - 1) / b + 1;
```

12.5 Type conversions

```java
// int[] to List<Integer> (no one-liner for primitives)
List<Integer> list = new ArrayList<>();
for (int num : nums) list.add(num);

// List<Integer> to int[]
int[] arr = new int[list.size()];
for (int i = 0; i < list.size(); i++) arr[i] = list.get(i);

// String <-> char[]
char[] chars = str.toCharArray();
String str = new String(chars);

// String <-> int
int num = Integer.parseInt("42");
String str = String.valueOf(42);

// int <-> long
long big = (long) smallInt;
int small = (int) bigLong;  // Careful: truncates if value exceeds int range

// Array to List (for object arrays only)
String[] arr = {"a", "b", "c"};
List<String> list = new ArrayList<>(Arrays.asList(arr));

// List to array
String[] arr = list.toArray(new String[0]);
```

12.6 Deep copy vs shallow copy

```java
List<Integer> path = new ArrayList<>();
path.add(1);
path.add(2);

List<List<Integer>> results = new ArrayList<>();
results.add(path);          // BAD: adds reference
path.add(3);
// results now contains [1, 2, 3], not [1, 2]

results.add(new ArrayList<>(path)); // GOOD: adds a copy


// for arrays
int[] copy = Arrays.copyOf(original, original.length);
int[] copy = original.clone();


// for 2d arrays
int[][] deepCopy = new int[matrix.length][];
for (int i = 0; i < matrix.length; i++) {
    deepCopy[i] = matrix[i].clone();
}
```

13 OOP basics for DSA

```java
// Linked list node
class ListNode {
    int val;
    ListNode next;

    ListNode(int val) {
        this.val = val;
        this.next = null;
    }

    // Overloaded constructor (used in some problems)
    ListNode(int val, ListNode next) {
        this.val = val;
        this.next = next;
    }
}
// Build list: 1 -> 2 -> 3 -> null
ListNode head = new ListNode(1, new ListNode(2, new ListNode(3)));

// Binary tree node
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;

    TreeNode(int val) {
        this.val = val;
    }
}

// Graph node with neighbors
class Node {
    int val;
    List<Node> neighbors;

    Node(int val) {
        this.val = val;
        this.neighbors = new ArrayList<>();
    }
}
```

```java
// Trie node
class TrieNode {
    TrieNode[] children;
    boolean isEndOfWord;

    TrieNode() {
        this.children = new TrieNode[26]; // For lowercase a-z
        this.isEndOfWord = false;
    }
}
```